

项目书基础版本

SketchSkill-AKG：面向昇腾 NPU 的技能驱动算子自动生成与硬件反馈优化系统

用于申请昇腾 910 NPU 云资源与模型 API 资源

赛题方向	基于AI/Agent的NPU算子自动生成
团队名称	算子炼金术师
负责人	于天池（帕多瓦大学博士生） & 郑遣俊（慕尼黑工业大学博士生）
联系方式	yu-tianchi@outlook.com
版本	基础版 v0.2
日期	2026 年 5 月

摘要

本项目面向比赛提供的昇腾 NPU 算子 Benchmark，拟在 AKG Kernel Agent 基础上构建一个 Sketch/Skill 驱动的算子自动生成与优化系统。项目不以“直接让大模型一次性写出底层 kernel”为唯一目标，而是将算子开发拆解为算子语义理解、NPU-aware Operator Sketch 规划、后端 DSL 代码生成、正确性验证、真实硬件 profiling、性能搜索和技能库沉淀等阶段。主路径采用 AKG Agents + Triton-Ascend 快速跑通 Benchmark；增强路径探索 TileLang-Ascend、Ascend C/CCE/TBE 与 MLIR/AscendNPU IR 的知识迁移。项目重点创新包括 NPU-aware Sketch、按算子模式组织的 Skill Library、正确性与性能双闭环 Agent、CUDA/Triton 经验向昇腾 NPU 迁移，以及训练无关的硬件反馈搜索。

—— 以下为正文 ——

一、团队介绍

本团队拟围绕 AI/Agent 驱动的高性能算子自动生成开展参赛工作，面向昇腾 910 NPU 后端，探索从算子问题描述、参考实现与 Benchmark 约束出发，自动生成可编译、可运行、可验证、可优化的 NPU 算子实现。团队成员关注编译系统、AI 芯片架构、IR/Pass 工程、算子调优、自动化验证与大模型辅助软件工程等方向，具备开展本项目所需的技术基础。

团队前期工作与能力储备包括：

- 理解昇腾 NPU 算子运行链路，包括 CANN/ACL Runtime、算子编译、运行、profiling 与日志分析。
- 关注 Ascend C、Triton、TileLang、MLIR 等算子语言与编译基础设施，理解 tile、访存、并行、同步、流水线等高性能 kernel 基本问题。
- 具备代码生成 Agent、RAG、Prompt/Skill 设计、自动化测试与错误修复流程的开发经验。
- 希望通过比赛沉淀一套可复用的 NPU 算子生成技能库和工程化闭环，而不仅是完成若干单点算子。

二、项目背景与研究现状

随着大模型结构、AI 加速硬件和编译器栈快速迭代，高质量算子的需求持续增长。传统算子开发依赖专家手工完成 kernel 编写、tile 策略设计、数据搬运优化、编译调试和性能分析，开发成本高且难以快速覆盖新模型、新 shape 与新后端。近年来，LLM 代码生成和 Agent 工作流为“AI 自动生成高性能算子”提供了新的技术路径。

从最新研究和开源生态看，算子自动生成正在从“单轮 LLM 直接写 kernel”转向“多阶段规划、验证驱动、profiling 驱动和技能库驱动”的闭环模式。KernelBench 将 LLM 写 CUDA/DSL kernel 作为系统评测任务，要求生成代码同时正确且高性能；AKG Kernel Agent 则进一步探索多 Agent、跨 DSL、跨硬件后端的 kernel 生成、迁移和调优。

在昇腾 NPU 场景下，直接让 LLM 生成 Ascend C 或复杂底层 kernel 难度更高，主要原因包括：Ascend C API 与 host-side tiling 约束强、公开高质量样例相对有限、硬件语义与 GPU 不完全一致、性能依赖 GM/UB 数据搬运和 copyin-compute-copyout 流水线，以及正确性与性能都必须通过真实 NPU 环境验证。AscendCraft、AscendOptimizer 等近期工作表明，轻量 DSL、结构化 lowering、硬件反馈搜索和 bad-to-good 经验库是提升 NPU kernel 生成质量的重要方向。

因此，本项目拟以比赛 Benchmark 为抓手，在 AKG Agents 已有能力之上，重点做更贴合昇腾 NPU 的结构化算子生成策略：使用 NPU-aware Operator Sketch 降低生成难度，使用按算子模式组织的 Skill Library 提升复用性，使用正确性与性能双闭环 Agent 提升 Pass@4 和性能表现。

三、项目定位与核心目标

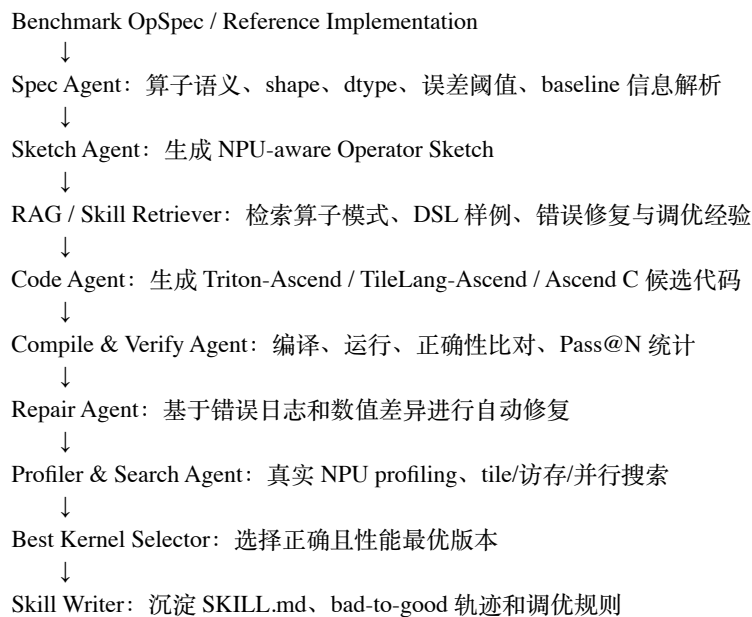
项目名称为“SketchSkill-AGK：面向昇腾 NPU 的技能驱动算子自动生成与硬件反馈优化系统”。项目核心目标是基于 AGK Kernel Agent 或其他开源方案，在昇腾 910 NPU 后端上实现从 Benchmark 算子描述到可执行 kernel 的自动生成、验证、优化和经验沉淀。

核心目标包括：

- 算子生成目标：实现从算子描述、参考实现、shape/dtype 约束到可执行 NPU kernel 的自动生成流程，主路径优先支持 Triton-Ascend。
- 正确性目标：在昇腾 910 环境中完成生成算子的编译和运行，与标准实现进行结果比对，统计 Pass@1、Pass@4、shape/dtype 覆盖率与失败原因。
- 性能目标：对正确性通过的 kernel 进行 latency、吞吐量和相对 baseline 的性能评估，引入 tile、并行映射、数据搬运和流水线搜索。
- AI 技术目标：构建 Spec Agent、Sketch Agent、Code Agent、Verifier Agent、Repair Agent、Profiler Agent 与 Search Agent 协作的双闭环 workflow。
- 资产沉淀目标：形成可复用的 SKILL.md、Prompt 模板、算子模式库、错误修复规则、调优规则、bad-to-good 轨迹和实验复现脚本。

四、总体技术路线

本项目采用“Benchmark 解析 + NPU-aware Sketch + 多后端代码生成 + 自动验证反馈 + 硬件反馈搜索 + Skill Library 写回”的总体路线。



初赛阶段主路径选择 AKG Agents + Triton-Ascend，原因是该路径与比赛代码仓和 Benchmark 最贴合，能够最快实现端到端闭环。增强路径将选择少量代表性算子尝试 TileLang-Ascend 或 Ascend C，以展示系统对多种算子语言和后端的扩展能力。

五、关键创新点

创新点一：NPU-aware Operator Sketch 表示

项目将算子生成拆分为“语义规划”和“代码实现”两个阶段。系统首先根据 Benchmark 描述生成 NPU-aware Sketch，显式表达算子类别、并行维度、tile 策略、GM/UB 数据搬运、copyin-compute-copyout 流水线、边界 mask、dtype 累加策略和性能调优旋钮；随后由 Code Agent 将 Sketch lowering 到 Triton-Ascend、TileLang-Ascend 或 Ascend C。该方式避免 LLM 直接面对复杂底层 API，降低幻觉和后端语义错误，提高编译成功率与正确性。

创新点二：按算子模式组织的 NPU Skill Library

项目将构建面向 elementwise、broadcast、reduction、transpose/layout、normalization、matmul-like 等算子类别的技能库。每个 Skill 不只是 Prompt，而是包含适用条件、shape/dtype 约束、Sketch 模板、tile 策略、访存策略、边界处理规则、常见编译错误、正确性错误、profiling 解释和 bad-to-good 优化轨迹。系统会把每轮生成、失败、修复和性能提升过程归档，使技能库随着 Benchmark 实验持续成长。

创新点三：正确性与性能双闭环 Agent

系统设计两个反馈闭环：第一层正确性闭环基于编译日志、运行错误和数值比对结果进行自动修复，提升 Pass@4；第二层性能闭环基于真实昇腾 910 profiling 结果，对 tile size、并行映射、数据搬运、UB 使用、循环结构和流水线策略进行搜索优化。系统会区分 API/语法/类型错误、shape/broadcast 错误、访存越界、归约结构错误和性能瓶颈，并将不同错误路由给不同 Agent，提高迭代效率。

创新点四：CUDA/Triton 经验向昇腾 NPU 迁移

针对 Ascend NPU 高质量公开样例相对不足的问题，项目引入 CUDA/Triton → NPU Sketch → Ascend 后端代码的迁移式生成策略。系统从成熟 CUDA/Triton 实现中抽取可迁移的算法结构和性能模式，例如 tile 划分、归约结构、数据复用、算子融合和内存访问模式；再映射到 NPU-aware Sketch，并生成 Triton-Ascend、TileLang-Ascend 或 Ascend C 代码。该方法利用 GPU 生态中的高质量经验，同时避免直接逐行翻译导致的后端语义不匹配。

创新点五：训练无关的硬件反馈搜索

考虑到比赛周期和 NPU 领域数据稀缺，项目不依赖大规模模型微调作为主路线，而采用多候选生成、自适应搜索和进化搜索。系统为每个算子生成多个 Sketch / Kernel 版本，在真实昇腾 910 环境中完成编

译、运行、正确性验证和性能 profiling，并根据 latency、Pass@N、shape 覆盖率和错误类型选择下一轮优化方向。该策略能够在有限样本下快速提高正确率和性能，并将搜索结果反哺 Skill Library。

六、系统模块设计

1. Benchmark 解析模块

读取比赛提供的 akg_kernels_bench_lite 任务，提取算子名称、输入输出 shape、dtype、参考实现、误差阈值、性能 baseline 和测试入口，并将其转成结构化 OpSpec。

2. Spec Agent

识别算子类别、输入输出关系、广播规则、归约维度、layout 变化、边界条件和数值精度要求，形成后续 Sketch 生成所需的语义摘要。

3. Sketch Agent

生成 NPU-aware Operator Sketch，包含 compute pattern、parallel axis、tile plan、memory plan、pipeline plan、boundary mask、accumulation dtype、performance knobs 等字段。

4. Skill Retriever / RAG 模块

从 AKG 文档、Triton-Ascend 样例、TileLang-Ascend 样例、Ascend C/CANN 文档、MLIR/AscendNPU IR 资料、历史成功/失败案例和 SKILL.md 中检索相关上下文。

5. Code Agent

根据 Sketch 和检索上下文生成候选 kernel。初赛主路径生成 Triton-Ascend；增强路径尝试 TileLang-Ascend 或 Ascend C。每个算子生成多个候选版本，用于 Pass@4 和性能筛选。

6. Compile & Verify Agent

自动调用编译、运行和正确性验证脚本，收集编译错误、运行错误、数值误差、失败 shape 和日志摘要。

7. Repair Agent

根据错误类型进行定向修复：语法/API 错误回传 Code Agent，shape/broadcast/tile 设计错误回传 Sketch Agent，数值误差错误触发 dtype、边界和归约策略检查。

8. Profiler & Search Agent

对正确性通过的 kernel 进行 latency 与 profiling 测试，通过 UCB、自适应搜索或进化搜索调节 tile、并行度、访存模式、double buffer 和循环结构。

9. Skill Writer

将成功实现、失败原因、修复过程、性能优化前后对比和可复用规则写入算子模式技能库，形成可持续增长的工程资产。

七、NPU-aware Operator Sketch 设计

NPU-aware Sketch 是本项目的核心中间表示，用于连接“算子语义理解”和“后端代码实现”。Sketch 不追求一次性成为完整 DSL，而是采用结构化 JSON/YAML 风格，便于 LLM 生成、检查、修复和检索。

对于不同算子类别，Sketch 模板不同。例如 `elementwise` 重点关注连续访存、向量化和尾部 mask；`reduction` 重点关注归约轴、累加 dtype、分块归约和并行归约；`transpose/layout` 重点关注读写连续性、tile 重排和 bank/conflict 风险；`matmul-like` 算子重点关注 M/N/K tile、数据复用、双缓冲和计算/搬运重叠。

八、Skill Library 设计

项目将建立一套面向昇腾 NPU 算子生成的结构化技能库，直接对应评分标准中的“AI辅助工作技能”。技能库不是简单 Prompt 集合，而是可被 Agent 检索和执行的工程 playbook。

```
skills/
elementwise/SKILL.md
broadcast/SKILL.md
reduction/SKILL.md
transpose_layout/SKILL.md
normalization/SKILL.md
matmul_like/SKILL.md
ascend_debug/SKILL.md
ascend_performance/SKILL.md
cuda_to_ascend_migration/SKILL.md
benchmark_evaluation/SKILL.md
```

每个 SKILL.md 固定包含以下内容：

- 适用算子类型与不适用边界；
- 输入输出 shape/dtype 约束；
- 推荐 NPU-aware Sketch 模板；
- 推荐 tile、core mapping、访存与流水线策略；
- Triton-Ascend / TileLang-Ascend / Ascend C 代码生成注意事项；
- 常见编译错误、运行错误和正确性错误；
- profiling 指标解释和性能瓶颈诊断；
- bad-to-good 示例：失败版本、修复过程、最终版本与性能变化。

技能库会随着 Benchmark 实验持续更新。例如，当某个 `reduction` 算子因尾部 mask 处理错误导致数值失败，Repair Agent 修复成功后，系统会将“错误特征—修复策略—适用条件”写入 `reduction/SKILL.md` 和错误案例库；当某个 tile 参数提升性能时，系统会将对应 shape、tile 和 latency 变化写入性能经验库。

九、多后端与算子语言策略

后端/语言	项目定位	使用方式
AKG Agents + Triton-Ascend	初赛主路径	优先跑通 Benchmark 自动生成、编译、验证和 Pass@4 统计。
TileLang-Ascend	增强路径	用于表达 tile/dataflow/pipeline，优先尝试 reduction、matmul-like、attention-like 或复杂访存算子。
Ascend C	高性能增强路径	针对少量代表性算子尝试直接生成或 Sketch-to-AscendC lowering，重点关注 host tiling 与 kernel 耦合。
CCE / TBE	知识与兼容路径	作为历史算子开发知识、API 语义和工程经验来源，用于 RAG 与 Skill 编写。
MLIR / AscendNPU IR	长期扩展路径	将 Sketch 设计为未来可映射到结构化 IR 的形式，支持更系统的 lowering 和优化。
CUDA / Triton	迁移知识来源	从成熟 GPU kernel 中抽取 tile、归约、融合、访存模式，转成 NPU Sketch 再生成 Ascend 后端实现。

十、正确性验证与性能验证方案

10.1 正确性验证

正确性验证以 Benchmark 标准实现为基准，在昇腾 910 环境中运行生成 kernel，并按 shape/dtype/误差阈值进行结果比对。系统将统计 Pass@1、Pass@4、各算子类别通过率、各错误类型占比和失败样例。

- Pass@1：单次生成候选中首个版本通过正确性验证的比例。
- Pass@4：每个算子生成 4 个候选版本，至少 1 个通过正确性验证的比例，作为比赛重点指标。
- 误差统计：记录 max error、mean error、rtol/atol 是否满足 Benchmark 要求。
- 失败原因：编译错误、运行错误、shape/dtype 错误、边界/mask 错误、数值误差、性能异常。

10.2 性能验证

性能验证在正确性通过后进行，记录 warmup、重复运行、平均/中位 latency、吞吐量和相对 baseline 的加速比。对于性能明显退化的版本，Profiler Agent 将分析可能瓶颈，包括 tile 不合理、访存不连续、UB 使用不足、并行度不足、循环结构不佳或数据搬运与计算未重叠。

10.3 搜索策略

初赛阶段采用训练无关的轻量搜索策略，包括多候选生成、UCB/自适应搜索、进化搜索和规则化参数枚举。搜索空间优先限制在可解释且可复用的性能旋钮上，例如 tile size、num cores、vector width、unroll factor、double buffer、parallel axis 和 boundary strategy。

十一、实施计划

阶段	时间	主要任务	阶段产出
阶段一：环境与 Benchmark 复现	初赛前期	申请并使用昇腾 910 资源；部署 AKG Agents；跑通 Benchmark baseline；整理算子清单。	环境说明、baseline 结果、Benchmark 分类表。
阶段二：端到端生成闭环	初赛中期	实现 OpSpec 解析、Sketch 生成、Triton-Ascend 代码生成、编译运行和正确性验证。	单算子到多算子的自动生成 demo，Pass@1/Pass@4 初步结果。
阶段三：Skill Library 与自动修复	初赛中后期	按算子类型建立 SKILL.md；实现错误日志摘要和 Repair Agent；沉淀失败修复案例。	技能库初版、错误分类表、bad-to-good 修复记录。
阶段四：性能搜索与多后端增强	初赛后期	对正确性通过 kernel 做 profiling 与搜索；选择代表性算子尝试 TileLang-Ascend 或 Ascend C。	性能对比表、优化前后案例、多后端尝试记录。
阶段五：项目提交与答辩准备	初赛收尾	整理代码、实验记录、复现文档、项目书完整版和 PR。	完整项目书、复现说明、PR、答辩材料。

十二、算力资源申请

本项目需要申请比赛提供的昇腾 910 NPU 云资源与模型 API 资源，原因如下：

- 比赛指定开发环境为昇腾 910，生成算子必须在目标硬件后端完成编译、运行和验证。
- 正确性验证依赖真实硬件执行结果，尤其是 dtype、边界、异步数据搬运和底层 kernel 行为，无法仅通过 CPU 模拟替代。
- 性能验证必须基于真实 NPU 进行，包括 latency、吞吐量、profiling、tile 搜索和与 baseline 的对比。
- Agent 生成流程需要多候选、多轮修复和多版本性能搜索，单个算子可能需要多次编译运行，因此需要稳定可用的 NPU 资源。
- Skill Library 的 bad-to-good 轨迹需要真实硬件反馈作为可信依据，尤其是性能优化经验不能只依赖静态分析。

资源用途包括：环境部署、Benchmark baseline 复现、生成 kernel 编译运行、Pass@N 统计、profiling、性能搜索、结果复现和答辩演示。

十三、预期成果与评价指标

初赛基础阶段以“跑通闭环、提升正确率、形成技能资产”为核心，性能优化以代表性案例为重点。预期成果包括：

- 面向昇腾 910 NPU 的 SketchSkill-AGK 原型系统；
- Benchmark 算子自动解析、生成、编译、运行、正确性验证和性能测试脚本；
- 若干 Benchmark 算子的 Pass@1、Pass@4、失败原因和性能对比结果；
- NPU-aware Operator Sketch 模板和算子类别分类器；
- 按算子模式组织的 SKILL.md、Prompt 模板、错误修复规则、调优规则和 bad-to-good 轨迹；
- 少量代表性算子的性能优化案例，展示优化前后 latency 或吞吐量变化；
- 完整项目说明、复现文档和 PR 提交。

阶段性指标建议如下：简单 elementwise/broadcast 类算子优先追求较高 Pass@4 和接近 baseline 的性能；reduction/transpose/layout 类算子重点展示 Sketch 与 Skill 对正确性修复和 tile 优化的帮助；复杂算子优先保证可编译、可运行和可复现实验记录，再逐步优化性能。

十四、风险分析与应对措施

风险	影响	应对措施
LLM 直接生成代码不稳定	编译失败、API 幻觉	采用 Sketch 中间层、RAG、Skill 约束和模板化生成。
正确性通过率不足	Pass@4 不达预期	多候选生成、错误日志分类、Repair Agent、按算子类型补充规则。
性能不及 baseline	评分受影响	先保正确性，再对代表性算子做 profiling 和轻量搜索，展示优化过程。
Ascend C/TileLang-Ascend 接入成本高	影响进度	主路径固定为 AKG + Triton-Ascend；其他后端仅作为增强案例。
NPU 资源有限	搜索轮次不足	缩小搜索空间，优先选高价值算子和高复用技能，记录每轮实验结果。
Benchmark 任务差异较大	模式库覆盖不足	先分类，按 elementwise/broadcast/reduction/layout/matmul-like 分层推进。

十五、初赛提交计划

本项目书基础版本用于申请比赛提供的昇腾 910 NPU 云资源和模型 API 资源。获得资源后，团队将立即开展环境搭建、Benchmark 复现和基础 Agent workflow 开发，并在初赛截止前持续完善项目代码、实验结果和项目书内容。

初赛阶段拟提交内容包括：

- 项目书完整版；
- 算子自动生成原型代码；
- Benchmark 运行、正确性验证和性能测试脚本；
- Pass@1、Pass@4 与性能对比结果；
- NPU-aware Sketch 模板；
- SKILL.md、Prompts、错误修复规则和性能调优规则；
- 项目复现说明文档；
- 向指定开源仓库提交的 PR。

十六、参考文献与相关开源项目

[1] MindSpore AKG. Auto Kernel Generator (AKG) README. GitHub / MindSpore official repository, 2026.

[2] MindSpore AKG Agents. AKG Agents README_CN: LLM 多 Agent 协作框架、Skill/Tools/SubAgent、LangGraph 工作流、Trace 系统与算子生成示例, 2026.

[3] Jinye Du et al. AKG Kernel Agent: A Multi-Agent Framework for Cross-Platform Kernel Synthesis. arXiv:2512.23424, 2025.

[4] Scaling Intelligence, Stanford. KernelBench: Can LLMs Write GPU Kernels? Blog and benchmark repository, 2024/2025.

[5] KernelBench authors. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517, 2025.

- [6] Zhongzhen Wen et al. AscendCraft: Automatic Ascend NPU Kernel Generation via DSL-Guided Transcompilation. arXiv:2601.22760, 2026.
- [7] AscendOptimizer authors. AscendOptimizer: Episodic Agent for Ascend NPU Operator Optimization. arXiv:2603.23566, 2026.
- [8] Tile-AI. TileLang: A Domain-Specific Language for High-Performance GPU/CPU Kernels. GitHub repository and documentation, 2025/2026.
- [9] Tile-AI. TileLang-Ascend: Tile Language for Huawei Ascend NPU. GitHub repository, 2025/2026.
- [10] Triton Project. Triton language and compiler documentation / GitHub repository.
- [11] LLVM MLIR Project. Linalg Dialect Documentation: structured ops, tiling, fusion and lowering.
- [12] Huawei / MindSpore. Ascend C Custom Operator Development and Usage. MindSpore tutorials.
- [13] Huawei Ascend. CANN Documentation: Ascend C Operator Development, Ascend C Best Practices, Profiling and AOE tools.
- [14] MindSpore Documentation. Glossary: CCE, CCE-C, TBE and Ascend-related terminology.
- [15] KrxGu. kernel-skills: Open source skill library for AI coding agents to write, optimize and debug high performance compute kernels. GitHub repository, 2026.
- [16] Kevin authors / Cognition. Kevin: Multi-Turn RL for Generating CUDA Kernels. arXiv:2507.11948, 2025.
- [17] Xiaoya Li et al. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning. arXiv:2507.14111 / OpenReview, 2025.
- [18] CUDA-Agent authors. Large-Scale Agentic RL for High-Performance CUDA Kernel Generation. Project page / arXiv, 2026.
- [19] Hugging Face Kernels / Blog. Custom CUDA kernels with agent skills and kernel-builder skills, 2026.